Protecting the ALU Logo on the Ethereum Blockchain

Assignment 1 – Cryptocurrency and Digital Assets · Golden Sash Munyankindi

Abstract

In this project I registered the official ALU logo on an Ethereum blockchain. The record keeps the logo's digital fingerprint, who registered it and when, so anyone can check if a logo file is real. I also made a second contract that splits ownership of the logo into 1,000,000 share tokens. The goal is to stop people from faking or falsely claiming the ALU logo.

Introduction

ALU's mission is to develop ethical, entrepreneurial leaders who will shape Africa and the world. For that to work, people need to trust anything that carries the ALU name. A fake logo on a fake certificate or course hurts that trust.

A logo is only a file, and files are easy to copy and edit. A normal database is not a good fix, because one administrator can change or delete records without anyone noticing. Copyright registration is also limited: it is slow, it depends on the country, and a stranger cannot quickly use it to check a file. A blockchain record is public, has a time stamp, and cannot be changed. Anyone can check it in seconds. If we store the logo's fingerprint on the blockchain, a changed copy will not match.

Blockchain Setup

I installed Node.js, made a new project, and installed Hardhat 2.29.1, the Hardhat Toolbox and OpenZeppelin Contracts 5.1.0. The file `hardhat.config.js` sets the Solidity version to 0.8.28. Hardhat does three jobs for me: it compiles my contracts, it runs my tests on a temporary test chain, and with `npx hardhat node` it starts a local Ethereum network at `http://127.0.0.1:8545`. That network gives me 20 test wallets, each with 10,000 fake ETH.

What is a local blockchain? It is a private Ethereum network that runs on my own computer. Developers use it instead of the real network because it is free, fast, and easy to reset. If I make a mistake, I lose nothing and nothing stays on a public chain.

What is a wallet address? It is a long code that starts with 0x, for example `0xf39F…2266`. It comes from a public key and works like an account number on the blockchain. It shows who owns ETH, NFTs and tokens, and its private key is used to sign transactions.

Smart Contract Explanation



The asset being protected: `alu-logo.png`, the ALU logo file that was hashed and registered.

How I made the logo hash. I downloaded the logo from the ALU website and saved it as `alu-logo.png`. Then I ran `shasum -a 256 alu-logo.png`. The deploy script and the tests make the same hash again using Node's `crypto` tool, so the value always comes from the real file. The hash is:
`727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560`

ALUAssetRegistry (ERC-721). This contract has a struct called `AssetMetadata` that holds the asset name, file type, content hash, the wallet that registered it, and the time. It has two mappings: one from token ID to metadata, and one from hash to true/false, which blocks duplicates. It has three functions:

- `registerAsset` – Takes a name, file type and hash. If the hash was already registered, it stops with the message "Asset already registered: duplicate content hash". If not, it adds one to the counter, saves the details, marks the hash as used, mints an NFT to the caller with `_safeMint`, sends an `AssetRegistered` event, and returns the new token ID. I save the data before minting, which is safer.
- `verifyLogoIntegrity` – Takes a token ID and a hash, and compares it with the saved hash. If they match it returns true and "Logo is authentic." If not it returns false and "Warning: logo does not match." It only reads data, so calling it costs no gas.
- `getAsset` – Takes a token ID and returns all the saved details for that token, so anyone can look them up.

What is a smart contract? It is a program stored on the blockchain and run by all the computers in the network. A normal program runs on a server owned by one company, and that company can change the code or the data whenever it wants. With a smart contract the rules are public, and nobody can quietly change them.

What is an NFT, and why ERC-721? An NFT is a unique token. No other token is the same, and each one has one owner. A logo is one unique thing that cannot be split, so ERC-721 fits it well. Each token ID is different and can carry its own details.

What is SHA-256? It is a function that turns any file into a fixed-length code, like a fingerprint. If you change even one pixel, the file's bytes change, and the new code comes out completely different. You also cannot work backwards from the code to the file. So a changed logo can never match the saved hash.

Why can't the data be changed? Blockchain data sits in blocks that are linked together by hashes, and thousands of computers keep a copy. Changing an old block would break all the blocks after it, and most of the network would reject it. My contract also has no function to edit or delete a record.

ALULogoToken (ERC-20). This token is called "ALU Logo Token" with the symbol ALUT. The total supply is fixed at 1,000,000, and the constructor gives all of it to the owner's wallet. It has two functions:

- `distributeShares(recipient, amount)` – Only the owner can use it. The amount must be more than zero. It moves that many tokens from the owner to the recipient.

- `ownershipPercentage(account)` – Returns a whole-number percentage: balance × 100 ÷ total supply. For example, 500,000 tokens gives 50.

The `deploy.js` script deploys the registry, registers the logo, and then deploys the token to the deployer's wallet.

Deployment. I deployed and tested both contracts on the local Hardhat network. The project also has a Sepolia testnet setup (see the README), so the same script can be used on a public testnet with a funded test wallet.

Tokenisation

Tokenisation means turning ownership of something into digital tokens on a blockchain, so it can be split into small parts, tracked, and passed on. For the ALU logo, ALU starts with all 1,000,000 ALUT tokens. It could then give shares to faculty who help build the brand, student representatives, the administration, and partner schools. This rewards people for their work and gives them a clear, provable stake.

ERC-721 and ERC-20 are different. ERC-721 is for one unique item, and ERC-20 is for many identical tokens that can be split and traded. That is why I used ERC-721 to register the one logo and ERC-20 for the shares of ownership.

If a faculty member holds 100,000 of the 1,000,000 ALUT tokens, they own 10% of the tokenised ownership of the logo. In practice this is a stake that is provable and can be transferred. ALU could attach things like voting or a share of income to it, but the token alone does not give legal rights. That would need a legal agreement.

Test Results

All eight tests pass: the logo registers and returns token ID 1; a duplicate hash is rejected; verifying with the right hash returns true; verifying with a wrong hash returns false; `getAsset` returns the right name and file type; the full supply is minted to the owner; `distributeShares` moves tokens; and `ownershipPercentage` returns the right value.

Conclusion

I learned how a hash works as a tamper-proof fingerprint, how ERC-721 and ERC-20 are used for different jobs, and how Hardhat and OpenZeppelin help me build and test contracts. If I had more time, I would store the logo file on IPFS and add a `tokenURI`, allow only approved ALU wallets to register assets, deploy to a public testnet, and build a web page for it in the next assignment.

GitHub Link

https://github.com/goldensash1/Cryptocurrency-and-Digital-Assets

Video

Video demonstration link: https://youtu.be/zuMnPCasyY8

Question 1: Exploring a Cryptocurrency Exchange

Step 1: Creating an account. To sign up on Coinbase you give an email, a password and a phone number. Then you give your full name, date of birth and home address, and upload a photo of a government ID. Sometimes you also take a selfie. Exchanges check who you are so that criminals cannot easily hide stolen money or pay for crime with crypto. This check is called KYC, which means Know Your Customer. In the United States it comes from the Bank Secrecy Act, and many other countries have similar anti-money-laundering laws based on rules from the Financial Action Task Force.

Step 2: Exploring the platform. The dashboard shows your balance and market prices. The wallet shows what you own, and the trading area is where you buy and sell. Here are three coins:

Bitcoin (BTC) was the first cryptocurrency. People use it to store value and send money without a bank. It is special because only 21 million coins will ever exist.

Ethereum (ETH) runs smart contracts. People use it to pay fees and to run apps, like NFTs. It is different because you can program on it, which is what I did in this project.

USD Coin (USDC) is a stablecoin. People use it to move dollars quickly or to keep value safe between trades. Unlike the other two, its price is meant to stay at one US dollar.

Step 3: Adding a payment method. In the payment settings you choose to add a payment method. For a debit card you enter the card number, expiry date, security code and billing address. For a bank account you enter your account details or log in through a linked bank. A debit card is faster, because the purchase happens almost at once. A bank transfer is usually cheaper but can take a few days. So use a card when you are in a hurry and a bank transfer when you want to save on fees.

Step 4: Simulating a purchase. A market order buys right away at the best price available. The trade is sure, but the exact price is not. A limit order buys only at the price you choose or better, so the price is sure, but it might never happen. I chose a market order because $100 is a small amount, and for a beginner it is simple and quick. On 1 November 2026, Coinbase's public price feed showed Ethereum at about $2,261. If the total fees were about 1.5%, then $1.50 goes to fees and the other $98.50 buys about 0.0436 ETH. I used the free version of the platform and did not buy anything, so I spent no money. The fee and the amount of ETH are my own estimates, not numbers from a real order.

Step 5: The transaction record. The history lists each transaction with its type, coin, amount, dollar value, fee, date and status. A transaction hash is a unique ID, a long string of letters and numbers, that the blockchain gives to a transaction. It is important because anyone can look it up on a block explorer to see that the transfer happened, when it happened, and that it was confirmed. Trades inside an exchange may only show in the exchange's own records, but a withdrawal to an outside wallet gets a hash.

Step 6: Security features. Two-factor authentication asks for a second code, from your phone or a security key, when you log in. It matters because a stolen password alone is not enough to get in. Withdrawal limits cap how

much can leave your account in a set time. They matter because crypto transfers usually cannot be undone, so a limit reduces the damage if someone breaks in.